



Instruction Sets Want to be Free!

Zhangxi Tan

Pure Storage Inc.

xtan@cs.berkeley.edu

www.riscv.org

December 17, 2016

(Slides modified from Krste's BSD meetup talk)





Why Instruction Set Architecture matters

- **Why can't Intel sell mobile chips?**
 - 99%+ of mobile phones/tablets based on ARM v7/v8 ISA
- **Why can't ARM partners sell servers?**
 - 99%+ of laptops/desktops/servers based on AMD64 ISA (over 95%+ built by Intel)
- **How can IBM still sell mainframes?**
 - IBM 360, oldest surviving ISA (50+ years)

***ISA is most important interface in computer system
where software meets hardware***

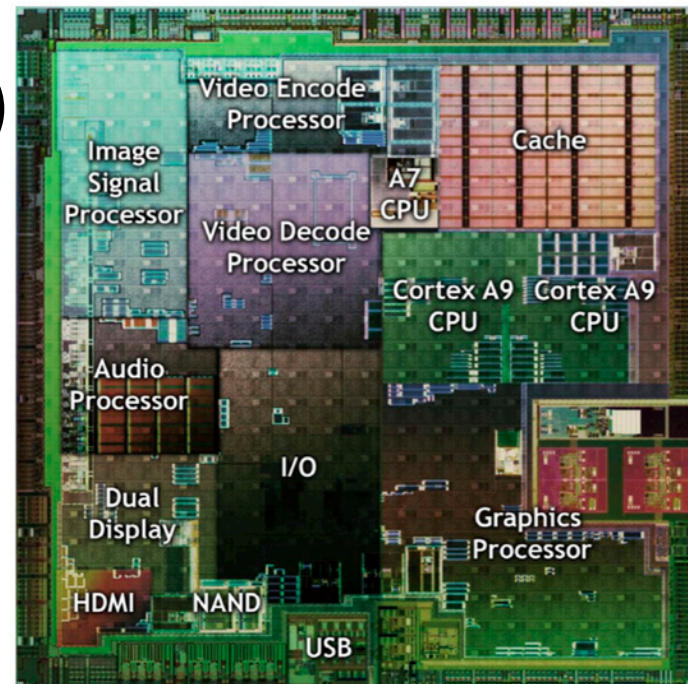
Open Software/Standards Work!

<i>Field</i>	<i>Standard</i>	<i>Free, Open Impl.</i>	<i>Proprietary Impl.</i>
Networking	Ethernet, TCP/IP	Many	Many
OS	Posix	Linux, FreeBSD	M/S Windows
Compilers	C	gcc, LLVM	Intel icc, ARMcc
Databases	SQL	MySQL, PostgreSQL	Oracle 12C, M/S DB2
Graphics	OpenGL	Mesa3D	M/S DirectX
ISA	??????	-----	x86, ARM, IBM360

- Why not successful free & open standards and free & open implementations, like other fields
- Dominant proprietary ISAs are **dismal** designs

Why so many ISAs on an SoC?

- Applications processor (usually ARM)
- Graphics processors
- Image processors
- Radio DSPs
- Audio DSPs
- Security processors
- Power-management processor
-
- Apps processor ISA too large for base accelerator ISA
- IP bought from different places, each proprietary ISA
- Home-grown ISA cores
- *Over a dozen ISAs on some SoCs – each with unique software stack*



NVIDIA Tegra SoC

Do we need all these different ISAs?

Must they be proprietary?

*What if there was one free and open
ISA everyone could use for everything?*



RISC-V Origins

- In 2010, after many years and many projects using MIPS, SPARC, and x86 as basis of research at Berkeley, time to choose ISA for next set of projects
- Obvious choices: x86 and ARM



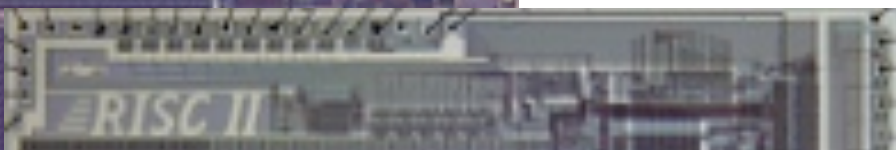
RISC-V Origin Story

- x86 impossible –IP issues, too complex
- ARM mostly impossible – no 64-bit, IP issues, complex
- So we started “3-month project” in summer 2010 to develop our own clean-slate ISA
 - Andrew Waterman, Yunsup Lee, Dave Patterson, Krste Asanovic principal designers
- Four years later, we released frozen base user spec
 - First public specification released in May 2011
 - Many tapeouts and several publications along the way

Why are outsiders complaining about changes to RISC-V in Berkeley classes?

Why is name RISC-V? (pronounced “risk-five”)

RISC-I



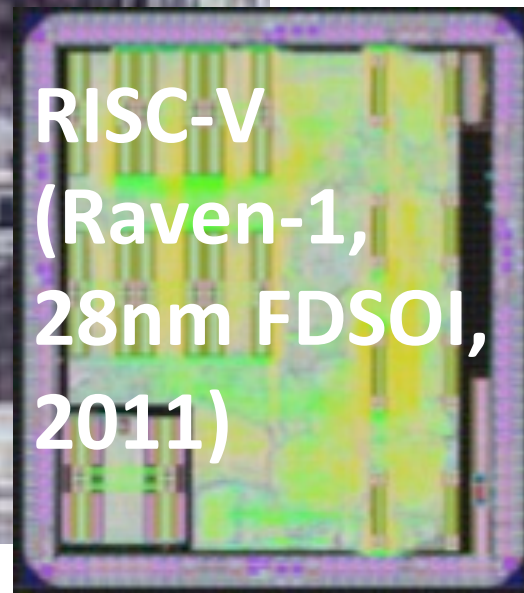
SOAR (aka RISC-III)



SPUR (aka RISC-IV)



RISC-V
(Raven-1,
28nm FDSOI,
2011)



Universal ISA Requirements

- Works well with existing software stacks, languages
- Is native hardware ISA, not virtual machine/ANDF
- Suits all sizes of processor, from smallest microcontroller to largest supercomputer
- Suits all implementation technologies, FPGA, ASIC, full-custom, future device technologies...
- Efficient for all microarchitecture styles: microcoded, in-order, decoupled, out-of-order, single-issue, superscalar, ...
- Supports extensive specialization to act as base for customized accelerators
- Stable: not changing, not disappearing

Why Didn't Other Open ISAs Take Off?

- **SPARC V8** - To its credit, Sun Microsystems made SPARC V8 an IEEE standard in 1994
 - Sun, Gaisler offered open-source cores
 - ISA now owned by Oracle
- **OpenRISC** - GNU open-source effort started in 1999, based on DLX from *Computer Architecture: AQA*
 - 64-bit ISA was in progress in 2010
 - Didn't separate Architecture and Implementation
- Competing in microprocessor era – now in SoC era
- Don't meet the needs of a universal ISA



What's Different about RISC-V?

- *Simple*
 - Far smaller than other commercial ISAs
- *Clean-slate design*
 - Clear separation between user and privileged ISA
 - Avoids μ architecture or technology-dependent features
- A *modular* ISA
 - Small standard base ISA
 - Multiple standard extensions
- Designed for *extensibility/specialization*
 - Variable-length instruction encoding
 - Vast opcode space available for instruction-set extensions
- *Stable*
 - Base and standard extensions are frozen
 - Additions via optional extensions, not new versions



RISC-V Base Plus Standard Extensions

- Four base integer ISAs
 - RV32E, RV32I, RV64I, RV128I
 - RV32E is 16-register subset of RV32I
 - Only <50 hardware instructions needed for base
- Standard extensions
 - M: Integer multiply/divide
 - A: Atomic memory operations (AMOs + LR/SC)
 - F: Single-precision floating-point
 - D: Double-precision floating-point
 - G = IMAFD, “General-purpose” ISA
 - Q: Quad-precision floating-point
- All the above are a fairly standard RISC encoding in a fixed 32-bit instruction format
- Above user-level ISA components frozen in 2014
 - Supported forever after

Base Integer Instructions (32 64 128)				
Category	Name	Fmt	RV{32 64 128}I Base	
Loads	Load Byte	I	LB	rd,rs1,imm
	Load Halfword	I	LH	rd,rs1,imm
	Load Word	I	L{W D Q}	rd,rs1,imm
	Load Byte Unsigned	I	LBU	rd,rs1,imm
	Load Half Unsigned	I	L{H W D}U	rd,rs1,imm
Stores	Store Byte	S	SB	rs1,rs2,imm
	Store Halfword	S	SH	rs1,rs2,imm
	Store Word	S	S{W D Q}	rs1,rs2,imm
Shifts	Shift Left	R	SLL{W D}	rd,rs1,rs2
	Shift Left Immediate	I	SLLI{W D}	rd,rs1,shamt
	Shift Right	R	SRL{W D}	rd,rs1,rs2
	Shift Right Immediate	I	SRLI{W D}	rd,rs1,shamt
	Shift Right Arithmetic	R	SRA{W D}	rd,rs1,rs2
	Shift Right Arith Imm	I	SRAI{W D}	rd,rs1,shamt
Arithmetic	ADD	R	ADD{W D}	rd,rs1,rs2
	ADD Immediate	I	ADDI{W D}	rd,rs1,imm
	SUBtract	R	SUB{W D}	rd,rs1,rs2
	Load Upper Imm	U	LUI	rd,imm
	Add Upper Imm to PC	U	AUIPC	rd,imm
Logical	XOR	R	XOR	rd,rs1,rs2
	XOR Immediate	I	XORI	rd,rs1,imm
	OR	R	OR	rd,rs1,rs2
	OR Immediate	I	ORI	rd,rs1,imm
	AND	R	AND	rd,rs1,rs2
	AND Immediate	I	ANDI	rd,rs1,imm
Compare	Set <	R	SLT	rd,rs1,rs2
	Set < Immediate	I	SLTI	rd,rs1,imm
	Set < Unsigned	R	SLTU	rd,rs1,rs2
	Set < Imm Unsigned	I	SLTIU	rd,rs1,imm
Branches	Branch =	SB	BEQ	rs1,rs2,imm
	Branch ≠	SB	BNE	rs1,rs2,imm
	Branch <	SB	BLT	rs1,rs2,imm
	Branch ≥	SB	BGE	rs1,rs2,imm
	Branch < Unsigned	SB	BLTU	rs1,rs2,imm
	Branch ≥ Unsigned	SB	BGEU	rs1,rs2,imm
Jump & Link	J&L	UJ	JAL	rd,imm
	Jump & Link Register	I	JALR	rd,rs1,imm
Synch	Synch thread	I	FENCE	
	Synch Instr & Data	I	FENCE.I	
System	System CALL	I	SCALL	
	System BREAK	I	SBREAK	
Counters	Read CYCLE	I	RDCYCLE	rd
	Read CYCLE upper Half	I	RDCYCLEH	rd
	Read TIME	I	RDTIME	rd
	Read TIME upper Half	I	RDTIMEH	rd
	Read INSTR RETired	I	RDINSTRET	rd
	Read INSTR upper Half	I	RDINSTRETH	rd

+14
Privileged

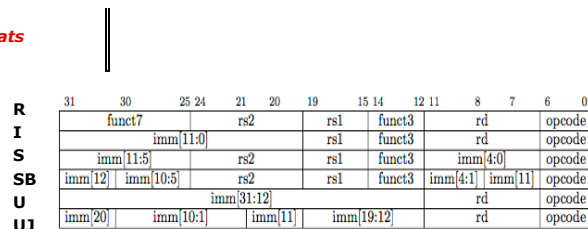
+ 8 for M

+ 34
for F, D, Q

+ 46 for C

+ 11 for A

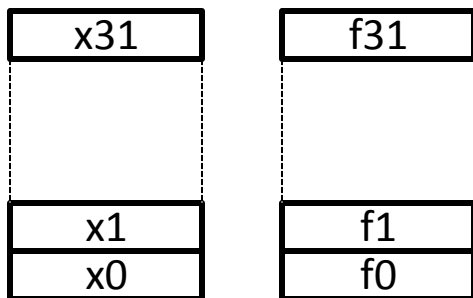
32-bit Instruction Formats





Proposed “V” Vector Extension State

Standard RISC-V scalar x and f registers



Vector configuration

CSR

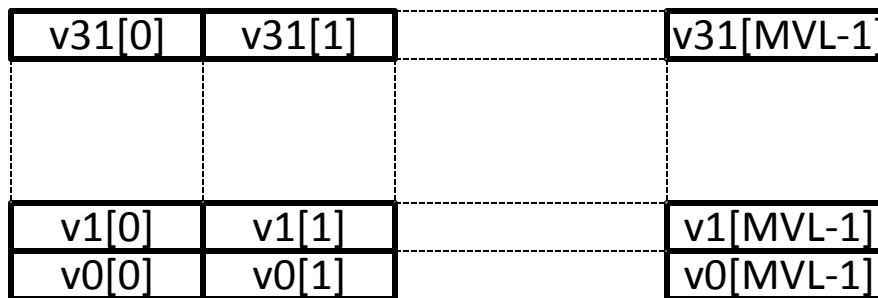
vcfg

Vector length

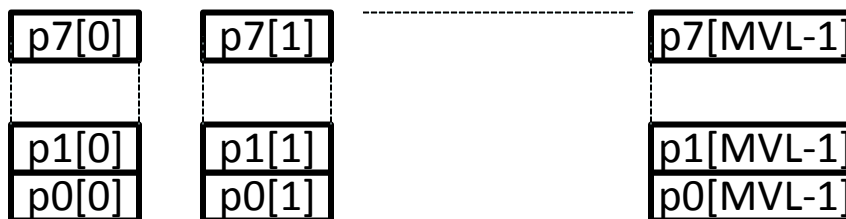
CSR

vlr

Up to 32 vector data registers, v0-v31, of at least 4 elements each, with variable bits/element (8,16,32,64,128)



MVL is maximum vector length, implementation and configuration dependent, but $MVL \geq 4$



8 vector predicate registers, with 1 bit per element

Does RISC-V support XXX?

Yes! If not, you can contribute

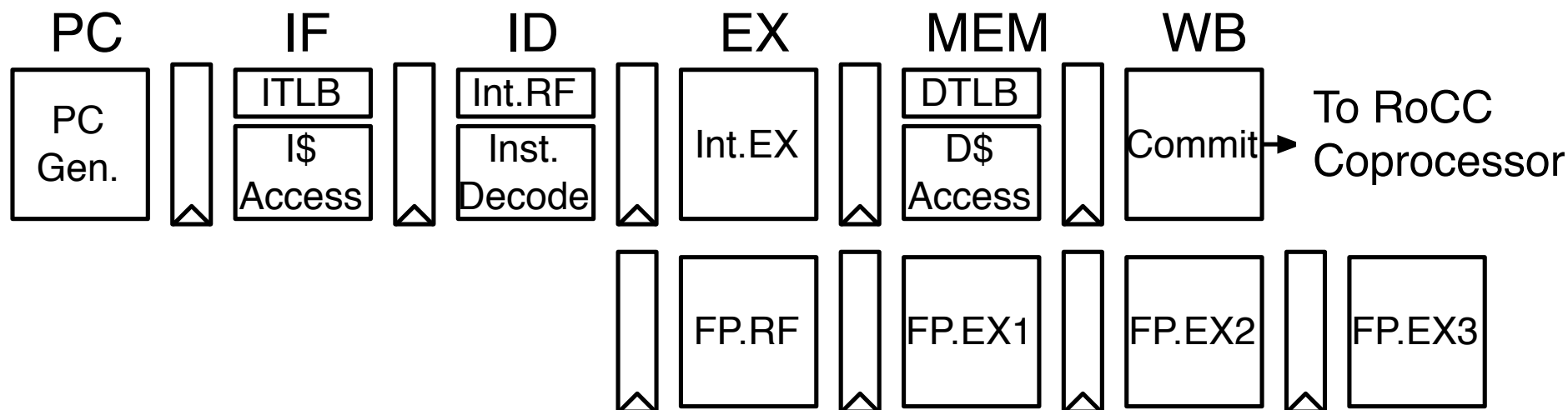
Current Software Landscape

- Linux LTS kernel support
 - Many **official** Linux distribution port in progress: Yocto/Poky, Gentoo, Debian
- BSD support
 - RISC-V support is **officially** in FreeBSD 11
 - Matt Thomas is working on the NetBSD port
- Support for primary open source toolchains
 - Binutils, GCC, clang/LLVM
 - Google Go
- Multiple software simulators/emulators
 - Spike, QEMU, Angel
- Java support: OpenJDK port (Berkeley)
- Bootloader/BIOS support:
 - Coreboot (Google) / UEFI (HP)

UC Berkeley RISC-V Core Generators

- **Rocket:** Family of In-order Cores
 - Supports 32-bit and 64-bit single-issue only
 - Dual-issue soon
 - Similar in spirit to ARM Cortex M-series and A5/A7/A53
- **BOOM:** Family of Out-of-Order Cores
 - Supports 64-bit single-, dual-, quad-issue
 - Similar in spirit to ARM Cortex A9/A15/A57

RISC-V Rocket In-Order Core



- 64-bit 5-stage single-issue in-order pipeline
- Design minimizes impact of long clock-to-output delays of compiler-generated RAMs
- 64-entry BTB, 256-entry BHT, 2-entry RAS
- MMU supports page-based virtual memory
- IEEE 754-2008-compliant FPU
 - Supports SP, DP fused multiply-adds with hardware support for subnormals
- Currently working on dual-issue in-order Rocket

ARM Cortex-A5 vs. RISC-V Rocket

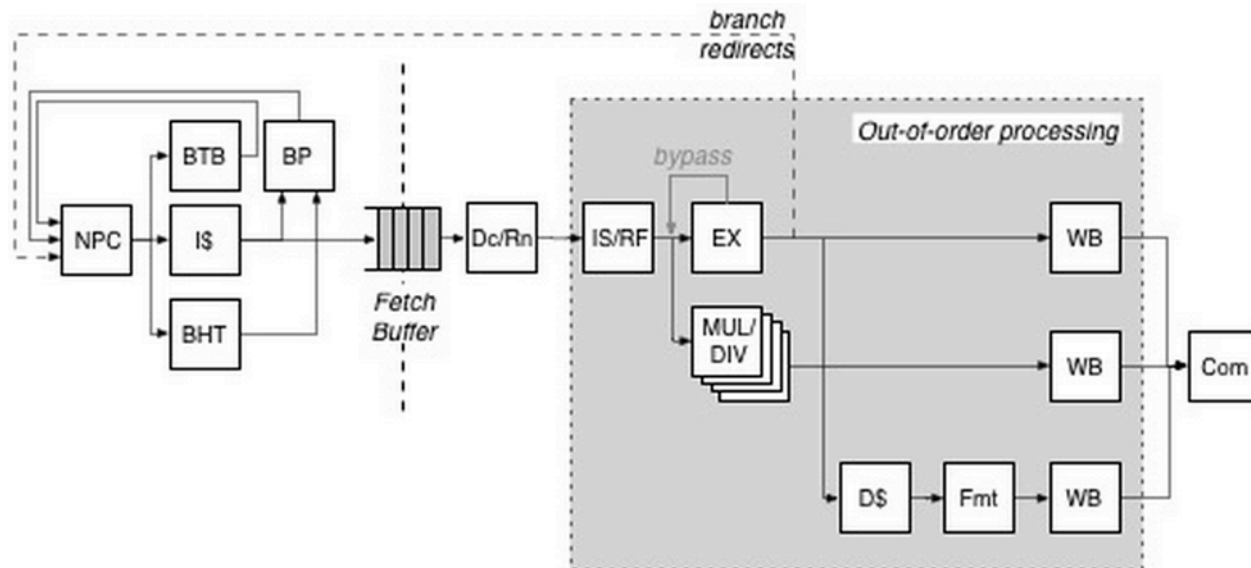
Category	ARM Cortex-A5	RISC-V Rocket
ISA	32-bit ARM v7	64-bit RISC-V v2
Architecture	Single-Issue In-Order	Single-Issue In-Order 5-stage
Performance	1.57 DMIPS/MHz	1.72 DMIPS/MHz
Process	TSMC 40GPLUS	TSMC 40GPLUS
Area w/o Caches	0.27 mm ²	0.14 mm ²
Area with 16K Caches	0.53 mm ²	0.39 mm ²
Area Efficiency	2.96 DMIPS/MHz/mm ²	4.41 DMIPS/MHz/mm ²
Frequency	>1GHz	>1GHz
Dynamic Power	<0.08 mW/MHz	0.034 mW/MHz

- PPA reporting conditions

- 85% utilization, use Dhrystone for benchmark, frequency/power at TT 0.9V 25C, all regular VT transistors

- 10% higher in DMIPS/MHz, 49% more area-efficient

RISC-V BOOM Out-of-Order Core



■ Baseline Design

- Superscalar (parameterizable widths)
- Full branch speculation (BTB/BHT/RAS)
- Load/store queue with store ordering
 - Loads execute fully OoO wrt stores, other loads
 - Store-data forwards to loads

■ Estimated 2GHz+ in 45nm (<30 FO4)

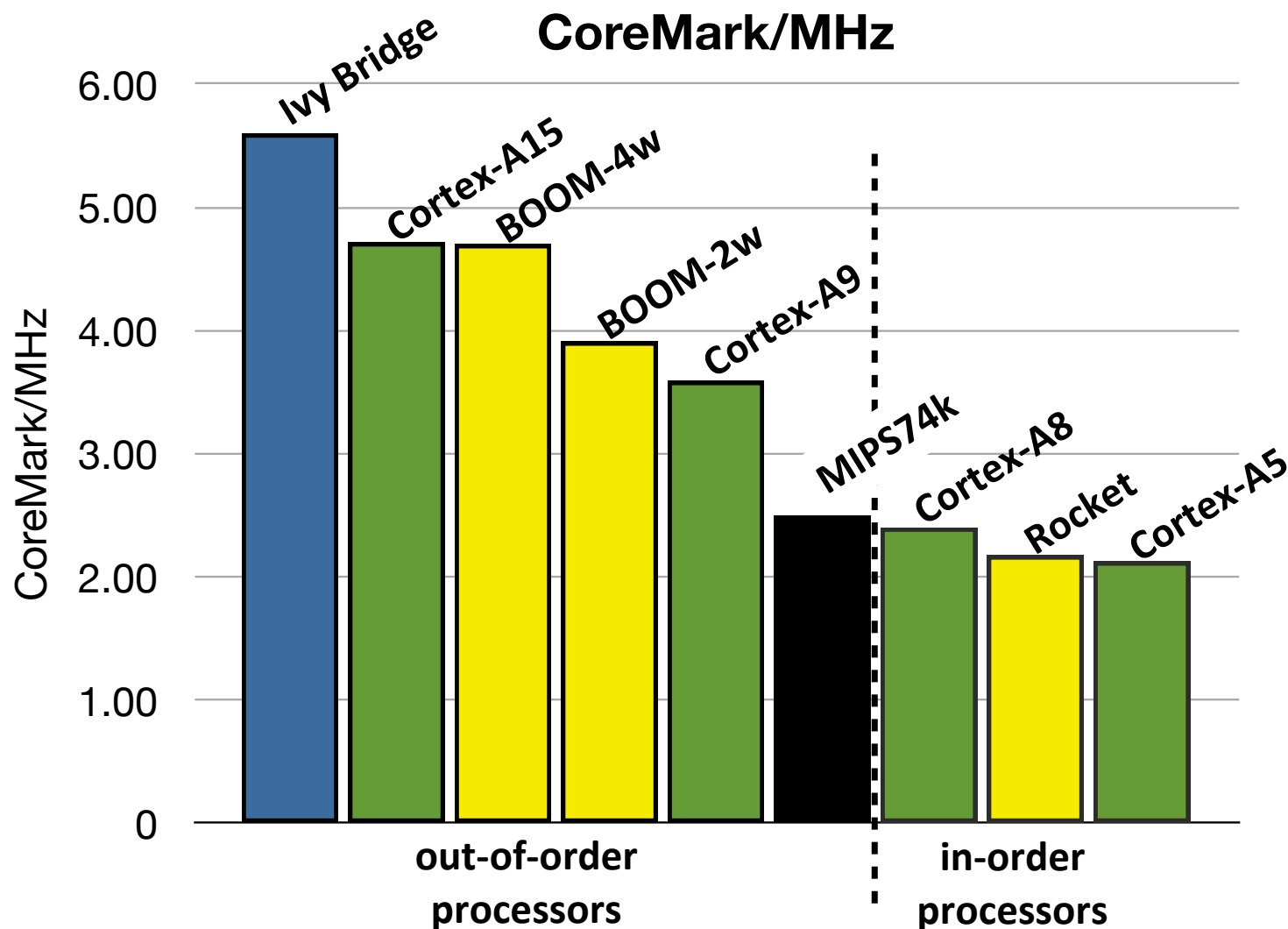
ARM Cortex-A9 vs. RISC-V BOOM

Category	ARM Cortex-A9	RISC-V BOOM-2w
ISA	32-bit ARM v7	64-bit RISC-V v2 (RV64G)
Architecture	2 wide, 3+1 issue Out-of-Order 8-stage	2 wide, 3 issue Out-of-Order 6-stage
Performance	3.59 CoreMarks/MHz	3.91 CoreMarks/MHz
Process	TSMC 40GPLUS	TSMC 40GPLUS
Area with 32K caches	2.5 mm ²	1.00 mm ²
Area efficiency	1.4 CoreMarks/MHz/mm ²	3.9 CoreMarks/MHz/mm ²
Frequency	1.4 GHz	1.5 GHz

Caveats: A9 includes NEON

BOOM is 64-bit, has IEEE-2008 fused mul-add

CoreMark Scores



See Chris Celio's "BOOM: Berkeley Out-of-Order Machine" talk from 2nd RISC-V Workshop for more details

RISC-V Outside Berkeley

- Adopted as “standard ISA” for **India**
 - IIT-Madras \$90M funding to build 6 different open-source RISC-V cores, from microcontrollers to servers
 - C-DAC \$45M funding to build 2GHz quad-core
- **LowRISC** project based in Cambridge, UK producing open-source RISC-V Rocket-based SoCs
 - Led by Raspberry Pi co-founder, privately funded
- **NVIDIA** selected RISC-V for on-chip microcontrollers
- Many companies developing RISC-V cores for use in tightly integrated hardware/software IP blocks
- First commercial RISC-V cores have already shipped!
 - Rumble Development Corp, for dental camera/imaging
- Multiple commercial silicon implementations should be for sale later this year

Who is contributing to RISC-V Now?

Universities	Companies	Open Source Projects
Berkeley University of Cambridge ETH Zurich Cornell North Carolina State University Texas Tech University University of Erlangen- Nuremberg	Bluespec Google LG Electronics BAE Systems SRI VectorBlox	Gentoo Debian FreeBSD NetBSD

- Landscape of 12/31/2016
 - Raspberry Pi like RISC-V platform to purchase (SiFive)
 - Upstreamed GNU toolchains
 - Upstreamed OS support
 - Finish Debian/RISC-V port
 - Planning Android support

Rambus : The ‘Linux’ of microprocessors



<http://www.gsaglobal.org/gsa-resources/publications/charting-a-new-course-for-semiconductors/>

Mass momentum on building out ecosystem



RISC-V Foundation

- Mission statement
 - “to standardize, protect, and promote the free and open RISC-V instruction set architecture and its hardware and software ecosystem for use in all computing devices.”
- Established as a 501(c)(6) non-profit corporation on August 3, 2015
- Rick O’Connor recruited as Executive Director
- First year, 41+ “founding” members. Additional members welcome:
 - Platinum (\$25K/year)
 - Gold (\$10K/year)
 - Silver (\$5K/year)

Foundation Principles

- The RISC-V ISA and related standards shall remain open and license-free to all parties. The standard specifications shall always be publicly available as an online download.
- The compatibility test suites shall always be publicly available as a source-code download.
- To protect the standard, only members of the Foundation in good standing can use “RISC-V” and associated trademarks for commercial products, and only for devices that pass the tests in the open-source compatibility suites maintained by the Foundation.

Foundation Members (41+)

Platinum:



Gold, Silver, Auditors:





Learning More about RISC-V

- Website **riscv.org** is primary resource
- Sign up for mailing lists/twitter at **riscv.org** to get announcements
- 1st RISC-V workshop January 14-15, 2015, Monterey
- 2nd RISC-V workshop June 29-30, 2015, UC Berkeley
- 3rd RISC-V workshop January 5-6, 2016, Oracle, CA
- 4th RISC-V Workshop July 12-13, 2016, MIT, MA
- 5th RISC-V Workshop, November 29-30, 2016, Google, Mountain View, CA
- All workshops sold out!
- Material from all workshops at **riscv.org**
- Next, 6th RISC-V Workshop, May 2017, Shanghai, China

RISC-V Architecture Roadmap

- **Nov 2016 - 5th workshop (@1.9.1 today)**
 - Agree/tweak plan, assign more leaders and doers
- **Feb 2017**
 - Priv-1.10.0
 - Debug spec ratified by Foundation
 - Calling convention fixed and documented
 - ELF format fixed and documented
 - M-mode/S-mode changes must be backwards-compatible after this date
- **March 2017**
 - Memory model changes must be backwards-compatible after this date
- **May 2017 - 6th workshop**
 - Priv-1.11.0
 - RV32EMAC RV32IMAFDQC RV64IMAFDQC ratified by Foundation
- **Aug 2017**
 - Priv-1.12.0
- **Nov 2017 - 7th workshop**
 - Priv-1.13.0 -> Priv-2.0 ratified?
 - V ratified by Foundation
 - Complete Linux/KVM, *BSD/Bhyve platform spec agreed

Summary: Why RISC-V?

- Free and open architecture, no proprietary lock-in
- Much simpler ISA than others (verification, security)
- Stable, will not change or disappear
- Enables better area/power/performance than other general-purpose ISAs at all design points
- Usable as base ISA for every core on SoC
- Readily and freely extensible

Modest RISC-V Project Goal

*Become the industry-standard ISA for all
computing devices*

“We are calling for volunteers” – David Patterson